

# Tarea S4.01. Creación de Base de Datos

## Patricia Proniewska

### Nivel 1

Descarga los archivos CSV, estudiales y diseña una base de datos con un esquema de estrella que contenga, al menos 4 tablas de las que puedas realizar las siguientes consultas:

Para crear la base de datos y poder realizar las consultas del ejercicio, necesito las tablas: company, american\_users, european\_users y credit\_card. Voy a crear la base de datos paso a paso y después explicar el diagrama/modelo resultante.

Creamos base de datos, primera tabla **company** e importamos datos del archivo csv.

```
29 -- crear base de datos:
30 • CREATE DATABASE transactions_sprint;
31 • USE transactions_sprint;
32
```

|    | Time     | Action                              | Response          |
|----|----------|-------------------------------------|-------------------|
| 23 | 00:21:59 | CREATE DATABASE transactions_sprint | 1 row(s) affected |

```
30 -- importar datos del archivo companies.csv en la tabla company mediante el comando SQL LOAD DATA LOCAL INFILE:
31 • LOAD DATA LOCAL INFILE '/Users/patrycjaproniewska/Documents/companies.csv'
32 INTO TABLE company
33 FIELDS TERMINATED BY ','
34 LINES TERMINATED BY '\n'
35 IGNORE 1 LINES
36 (company_id, company_name, phone, email, country, website);
37
```

|    | Time     | Action                                                                                                                        | Response                                              |
|----|----------|-------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------|
| 26 | 00:26:03 | LOAD DATA LOCAL INFILE '/Users/patrycjaproniewska/Documents/companies.csv' INTO TABLE company FIELDS TERMINATED BY ',' LIN... | 100 row(s) affected Records: 100 Deleted: 0 Skippe... |

```
23 -- crear tabla 'company':
24 • CREATE TABLE company (
25     company_id VARCHAR(15) PRIMARY KEY,
26     company_name VARCHAR(150),
27     phone VARCHAR(15),
28     email VARCHAR(100) UNIQUE,
29     country VARCHAR(100),
30     website VARCHAR(150)
31 );
32
```

|    | Time     | Action                                                                                                              | Response          |
|----|----------|---------------------------------------------------------------------------------------------------------------------|-------------------|
| 25 | 00:25:02 | CREATE TABLE company ( company_id VARCHAR(15) PRIMARY KEY, company_name VARCHAR(150), phone VARCHAR(15), email V... | 0 row(s) affected |

Creamos tabla **credit\_card** e importamos datos del archivo csv:

```
42 -- crear tabla 'credit_card':
43 • CREATE TABLE credit_card (
44     id VARCHAR(20) PRIMARY KEY,
45     user_id VARCHAR(20),
46     iban VARCHAR(50),
47     pan VARCHAR(20),
48     pin VARCHAR(4),
49     cvv VARCHAR(4),
50     track1 VARCHAR(100),
51     track2 VARCHAR(100),
52     expiring_date VARCHAR(20)
53 );
54
```

00% 3:53

Action Output

|    | Time     | Action                                                                                                                  | Response          |
|----|----------|-------------------------------------------------------------------------------------------------------------------------|-------------------|
| 28 | 00:30:55 | CREATE TABLE credit_card ( id VARCHAR(20) PRIMARY KEY, user_id VARCHAR(20), iban VARCHAR(50), pan VARCHAR(20), pin V... | 0 row(s) affected |

```
55 -- importar datos del archivo companies.csv en la tabla company mediante el comando SQL LOAD DATA LOCAL INFILE:
56 • LOAD DATA LOCAL INFILE '/Users/patrycjaproniewska/Documents/credit_cards.csv'
57 INTO TABLE credit_card
58 FIELDS TERMINATED BY ','
59 LINES TERMINATED BY '\n'
60 IGNORE 1 LINES
61 (id, user_id, iban, pan, pin, cvv, track1, track2, expiring_date);
62
```

100% 67:61

Action Output

|    | Time     | Action                                                                                                                           | Response                                              |
|----|----------|----------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------|
| 29 | 00:33:48 | LOAD DATA LOCAL INFILE '/Users/patrycjaproniewska/Documents/credit_cards.csv' INTO TABLE credit_card FIELDS TERMINATED BY ','... | 5000 row(s) affected Records: 5000 Deleted: 0 Skip... |

```
63 -- verificar:
64 • SELECT *
65 FROM credit_card;
```

100% 18:65

Result Grid

|          | id  | user_id                  | iban             | pan  | pin | cvv                                  | track1   | track2 | expiring_date |
|----------|-----|--------------------------|------------------|------|-----|--------------------------------------|----------|--------|---------------|
| CcS-4859 | 278 | XX7826930491423553609370 | 8861684536289642 | 4983 | 277 | %B8861684536289642=2502101761665371? | 11/26/26 |        |               |
| CcS-4860 | 279 | XX5559590368835304645299 | 2481155515498459 | 6876 | 661 | %B2481155515498459=2602101514414395? | 07/27/27 |        |               |
| CcS-4861 | 280 | XX2035182877195191627307 | 1308930301149557 | 5710 | 398 | %B1308930301149557=2805101751305028? | 04/25/26 |        |               |
| CcS-4862 | 281 | XX4774721462463645409758 | 6715617009807829 | 4042 | 174 | %B6715617009807829=2210101702370428? | 11/27/26 |        |               |
| CcS-4863 | 282 | XX1476829664245046207111 | 3140879819451394 | 5969 | 449 | %B3140879819451394=3210101158648599? | 12/27/29 |        |               |
| CcS-4864 | 283 | XX6380298893385731196159 | 5793672133649114 | 8481 | 139 | %B5793672133649114=2306101806367101? | 02/28/26 |        |               |
| CcS-4865 | 284 | XX7085078596101025280599 | 5101552687251312 | 7847 | 903 | %B5101552687251312=3010101664862710? | 11/25/28 |        |               |
| CcS-4866 | 285 | XX4792859188206596406839 | 8080768801072613 | 9271 | 961 | %B8080768801072613=2810101715885695? | 02/28/25 |        |               |
| CcS-4867 | 286 | XX6038298816319374853717 | 7761849537661098 | 4820 | 862 | %B7761849537661098=2206101710411371? | 11/30/25 |        |               |

credit\_card 2

Action Output

|    | Time     | Action                    | Response             |
|----|----------|---------------------------|----------------------|
| 30 | 00:35:32 | SELECT * FROM credit_card | 5000 row(s) returned |

Creamos tabla **europ**ean\_users e importamos datos del archivo csv:

```
67 -- crear tabla 'european_users'
68 -- he añadido la columna 'continent' para conservar la información geográfica
69 -- y poder realizar posteriormente una unión (UNION) con la tabla 'american_users':
70 CREATE TABLE european_users (
71     id INT PRIMARY KEY,
72     name VARCHAR(100),
73     surname VARCHAR(100),
74     phone VARCHAR(150),
75     email VARCHAR(150),
76     birth_date VARCHAR(100),
77     country VARCHAR(150),
78     city VARCHAR(150),
79     postal_code VARCHAR(100),
80     address VARCHAR(255),
81     continent VARCHAR(50) DEFAULT 'Europe'
82 );
83
```

|    | Time     | Action                                                                                                            | Response          |
|----|----------|-------------------------------------------------------------------------------------------------------------------|-------------------|
| 31 | 00:38:19 | CREATE TABLE european_users ( id INT PRIMARY KEY, name VARCHAR(100), surname VARCHAR(100), phone VARCHAR(150),... | 0 row(s) affected |

```
84 -- importar datos del archivo european_users.csv en la tabla european_users mediante el comando SQL LOAD DATA LOCAL INFILE:
85 LOAD DATA LOCAL INFILE '/Users/patrycjaaproniewska/Documents/european_users.csv'
86 INTO TABLE european_users
87 FIELDS TERMINATED BY ','
88 ENCLOSED BY '"'
89 LINES TERMINATED BY '\n'
90 IGNORE 1 LINES
91 (id, name, surname, phone, email, birth_date, country, city, postal_code, address);
92
```

|    | Time     | Action                                                                                                                        | Response                                              |
|----|----------|-------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------|
| 32 | 00:39:13 | LOAD DATA LOCAL INFILE '/Users/patrycjaaproniewska/Documents/european_users.csv' INTO TABLE european_users FIELDS TERMINAT... | 3990 row(s) affected Records: 3990 Deleted: 0 Skip... |

He creado una columna llamada '**continent**', ya que más adelante planeo unir **american\_users** y **european\_users** en una sola tabla **users**, para agilizar las consultas y conservar la información del continente.

```
93 -- verificar:
94 SELECT *
95 FROM european_users;
```

|    | Time     | Action                       | Response             |
|----|----------|------------------------------|----------------------|
| 33 | 00:40:09 | SELECT * FROM european_users | 3990 row(s) returned |

| id  | name     | surname | phone           | email                              | birth_date   | country        | city       | postal_code | address                       | continent |
|-----|----------|---------|-----------------|------------------------------------|--------------|----------------|------------|-------------|-------------------------------|-----------|
| 153 | Keegan   | Pugh    | (016977) 3851   | sodales.nisi@aol.org               | Jul 27, 1994 | United Kingdom | London     | EC1A 1BB    | Ap #312-5898 Consectetur St.  | Europe    |
| 154 | Cooper   | Bullock | (021) 2521 6627 | et@outlook.net                     | Nov 2, 1986  | United Kingdom | Manchester | M1 1AE      | 872-1866 Pedo Rd.             | Europe    |
| 155 | Joshua   | Russell | 055 4409 5286   | justo.nec.ante@outlook.edu         | Jan 23, 1984 | United Kingdom | Manchester | M1 1AE      | Ap #285-4727 Auctor. Av.      | Europe    |
| 156 | Remedios | Case    | 055 3114 1566   | mollis.phasellus.libero@aol.com    | Oct 9, 1994  | United Kingdom | Liverpool  | L1 1AA      | 479-3690 Turpis Road          | Europe    |
| 157 | Philip   | Carey   | 0800 640 6251   | phasellus@yahoo.net                | Oct 10, 1992 | United Kingdom | Manchester | M1 1AE      | 196-1103 Quisque Street       | Europe    |
| 158 | Fatima   | Dyer    | 0800 1111       | adipiscing@google.org              | Dec 24, 1988 | United Kingdom | Birmingham | B1 1AA      | Ap #435-7194 Scelerisque, St. | Europe    |
| 159 | Kylynn   | Acevedo | 056 5727 9602   | dignissim.lacus.aliquam@google.org | May 10, 2000 | United Kingdom | Liverpool  | L1 1AA      | 871-3506 Lectus. Ave          | Europe    |
| 160 | Lael     | Moody   | 07123 850737    | nunc.sed@aol.org                   | Nov 19, 1987 | United Kingdom | London     | EC1A 1BB    | Ap #633-810 Purus. Road       | Europe    |
| 161 | Nora     | Reeves  | (01692) 29410   | ac@aol.com                         | Dec 24, 1989 | United Kingdom | London     | EC1A 1BB    | Ap #956-407 Et St.            | Europe    |

Creamos tabla **american\_users** e importamos datos del archivo csv:

```
97 -- crear tabla 'american_users':
98 -- he añadido la columna 'continent' para conservar la información geográfica
99 -- y poder realizar posteriormente una unión (UNION) con la tabla 'european_users':
100 • CREATE TABLE american_users (
101     id INT PRIMARY KEY,
102     name VARCHAR(100),
103     surname VARCHAR(100),
104     phone VARCHAR(150),
105     email VARCHAR(150),
106     birth_date VARCHAR(100),
107     country VARCHAR(150),
108     city VARCHAR(150),
109     postal_code VARCHAR(100),
110     address VARCHAR(255),
111     continent VARCHAR(50) DEFAULT 'America'
112 );
113
```

100% 3:112

Action Output

|    | Time     | Action                                                                                                            | Response          |
|----|----------|-------------------------------------------------------------------------------------------------------------------|-------------------|
| 34 | 00:43:53 | CREATE TABLE american_users ( id INT PRIMARY KEY, name VARCHAR(100), surname VARCHAR(100), phone VARCHAR(150),... | 0 row(s) affected |

```
114 -- importar datos del archivo american_users.csv en la tabla american_users
115 -- mediante el comando SQL LOAD DATA LOCAL INFILE:
116 • LOAD DATA LOCAL INFILE '/Users/patrycjaaproniewska/Documents/american_users.csv'
117 INTO TABLE american_users
118 FIELDS TERMINATED BY ','
119 ENCLOSED BY '"'
120 LINES TERMINATED BY '\n'
121 IGNORE 1 LINES
122 (id, name, surname, phone, email, birth_date, country, city, postal_code, address);
123
```

100% 84:122

Action Output

|    | Time     | Action                                                                                                                         | Response                                               |
|----|----------|--------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------|
| 35 | 00:44:50 | LOAD DATA LOCAL INFILE '/Users/patrycjaaproniewska/Documents/american_users.csv' INTO TABLE american_users FIELDS TERMINATE... | 1010 row(s) affected Records: 1010 Deleted: 0 Skipp... |

```
124 -- verificar:
125 • SELECT *
126 FROM american_users;
```

100% 21:126

| Result Grid                                          |    |        |          |                |                                     |              |               |              |             |                          |           |
|------------------------------------------------------|----|--------|----------|----------------|-------------------------------------|--------------|---------------|--------------|-------------|--------------------------|-----------|
| Filter Rows: Search Edit: Export/Import: Fetch rows: |    |        |          |                |                                     |              |               |              |             |                          |           |
|                                                      | id | name   | surname  | phone          | email                               | birth_date   | country       | city         | postal_code | address                  | continent |
| 3                                                    | 3  | Ciaran | Harrison | (522) 598-1365 | interdum.feugiat@aol.org            | Apr 29, 1998 | United States | Houston      | 77001       | 736-2063 Tellus St.      | America   |
| 4                                                    | 4  | Howard | Stafford | 1-411-740-3269 | ornare.egestas@icloud.edu           | Feb 18, 1989 | United States | Phoenix      | 85001       | Ap #545-2244 Erat. Rd.   | America   |
| 5                                                    | 5  | Hayla  | Pierce   | 1-554-541-2077 | et.malesuada.fames@hotmail.org      | Sep 26, 1998 | United States | Philadelphia | 19101       | 341-2821 Ultrices Av.    | America   |
| 6                                                    | 6  | Joel   | Tyson    | (718) 288-8020 | gravida.nunc.sed@yahoo.ca           | Oct 15, 1989 | United States | San Jose     | 95101       | 888-2799 Amet Street     | America   |
| 7                                                    | 7  | Rafael | Jimenez  | (817) 689-0478 | eget@outlook.ca                     | Dec 4, 1981  | United States | Chicago      | 60601       | 8627 Malesuada Rd.       | America   |
| 8                                                    | 8  | Nissim | Franks   | (692) 157-3469 | egestas.aliquam.fringilla@google.ca | Aug 1, 1993  | United States | New York     | 10001       | Ap #251-7144 Integer St. | America   |
| 9                                                    | 9  | Mannix | Mcclain  | (590) 883-2184 | aliquam.nisl@outlook.com            | Jan 24, 1987 | United States | San Antonio  | 78201       | 647-3080 Lacus. St.      | America   |
| 10                                                   | 10 | Robert | McCarthy | (324) 746-6771 | fermentum@protonmail.com            | Apr 30, 1984 | United States | San Jose     | 95101       | P.O. Box 773             | America   |
| 11                                                   | 11 | Joan   | Baird    | (981) 429-8106 | et@outlook.net                      | Feb 25, 1990 | United States | Los Angeles  | 90001       | P.O. Box 687             | America   |

american\_users 4

Action Output

|    | Time     | Action                       | Response             |
|----|----------|------------------------------|----------------------|
| 36 | 00:45:50 | SELECT * FROM american_users | 1010 row(s) returned |

Creamos tabla **users** como union de **european\_users** y **american\_users**:

```
131 • CREATE TABLE users (  
132     id INT PRIMARY KEY,  
133     name VARCHAR(100),  
134     surname VARCHAR(100),  
135     phone VARCHAR(150),  
136     email VARCHAR(150),  
137     birth_date VARCHAR(100),  
138     country VARCHAR(150),  
139     city VARCHAR(150),  
140     postal_code VARCHAR(100),  
141     address VARCHAR(255),  
142     continent VARCHAR(50)  
143 );|  
144
```

100% 3:143

Action Output

|    | Time     | Action                                                                                                           | Response          |
|----|----------|------------------------------------------------------------------------------------------------------------------|-------------------|
| 37 | 00:47:42 | CREATE TABLE users ( id INT PRIMARY KEY, name VARCHAR(100), surname VARCHAR(100), phone VARCHAR(150), email V... | 0 row(s) affected |

```
145 -- unir tablas: 'european_user' y 'american_user':  
146 • INSERT INTO users  
147 SELECT * FROM european_users  
148 UNION ALL  
149 SELECT * FROM american_users;  
150
```

100% 30:149

Action Output

|    | Time     | Action                                                                                | Response                                            |
|----|----------|---------------------------------------------------------------------------------------|-----------------------------------------------------|
| 38 | 00:48:45 | INSERT INTO users SELECT * FROM european_users UNION ALL SELECT * FROM american_users | 5000 row(s) affected Records: 5000 Duplicates: 0... |

```
156 • SELECT *  
157 FROM users;|  
158
```

100% 12:157

Result Grid

|     | id      | name      | surname | phone          | email                               | birth_date   | country       | city         | postal_code | address                  | continent |
|-----|---------|-----------|---------|----------------|-------------------------------------|--------------|---------------|--------------|-------------|--------------------------|-----------|
| ▶ 1 | Zeus    | Gamble    |         | 1-282-581-0551 | interdum.enim@protonmail.edu        | Nov 17, 1985 | United States | New York     | 10001       | 348-7818 Sagittis St.    | America   |
| 2   | Garrett | Mcconnell |         | (718) 257-2412 | integer.vitae.nibh@protonmail.org   | Aug 23, 1992 | United States | Philadelphia | 19101       | 903 Sit Ave              | America   |
| 3   | Ciaran  | Harrison  |         | (522) 598-1365 | interdum.feugiat@aol.org            | Apr 29, 1998 | United States | Houston      | 77001       | 736-2063 Tellus St.      | America   |
| 4   | Howard  | Stafford  |         | 1-411-740-3269 | ornare.egestas@icloud.edu           | Feb 18, 1989 | United States | Phoenix      | 85001       | Ap #545-2244 Erat. Rd.   | America   |
| 5   | Hayfa   | Pierce    |         | 1-554-541-2077 | et.malesuada.fames@hotmail.org      | Sep 26, 1998 | United States | Philadelphia | 19101       | 341-2821 Ultrices Av.    | America   |
| 6   | Joel    | Tyson     |         | (718) 288-8020 | gravida.nunc.sed@yahoo.ca           | Oct 15, 1989 | United States | San Jose     | 95101       | 888-2799 Amet Street     | America   |
| 7   | Rafael  | Jimenez   |         | (817) 689-0478 | eget@outlook.ca                     | Dec 4, 1981  | United States | Chicago      | 60601       | 8627 Malesuada Rd.       | America   |
| 8   | Nissim  | Franks    |         | (692) 157-3469 | egestas.aliquam.fringilla@google.ca | Aug 1, 1993  | United States | New York     | 10001       | Ap #251-7144 Integer St. | America   |
| 9   | Mannix  | Mccolain  |         | (590) 883-2184 | aliquam.nisi@outlook.com            | Jan 24, 1987 | United States | San Antonio  | 78201       | 647-3080 Lacus. St.      | America   |

users 6

Action Output

|    | Time     | Action              | Response             |
|----|----------|---------------------|----------------------|
| 40 | 00:50:36 | SELECT * FROM users | 5000 row(s) returned |

```
151 -- verificar:  
152 • SELECT continent, COUNT(*)  
153 FROM users  
154 GROUP BY continent;  
155
```

100% 20:154

Result Grid

|   | continent | COUNT(*) |
|---|-----------|----------|
| ▶ | America   | 1010     |
|   | Europe    | 3990     |

Result 7

Action Output

|    | Time     | Action                                                   | Response          |
|----|----------|----------------------------------------------------------|-------------------|
| 41 | 00:51:00 | SELECT continent, COUNT(*) FROM users GROUP BY continent | 2 row(s) returned |

Borramos tablas **europaan\_user** y **american\_user** porque los datos fueron consolidados en la tabla **users**:

```
160 -- borrar tablas 'europaan_user' y 'american_user':
161 • DROP TABLE IF EXISTS europaan_users;
```

|      | Time     | Action                              | Response          |
|------|----------|-------------------------------------|-------------------|
| ✓ 42 | 00:52:41 | DROP TABLE IF EXISTS europaan_users | 0 row(s) affected |

```
160 -- borrar tablas 'europaan_user' y 'american_user':
161 • DROP TABLE IF EXISTS europaan_users;
162 • DROP TABLE IF EXISTS american_users;
```

|      | Time     | Action                              | Response          |
|------|----------|-------------------------------------|-------------------|
| ✓ 43 | 00:52:58 | DROP TABLE IF EXISTS american_users | 0 row(s) affected |

Creamos tabla **transactions** e importamos datos del archivo csv:

```
166 -- crear tabla 'transactions':
167 • CREATE TABLE transactions (
168     id VARCHAR(255) PRIMARY KEY,
169     card_id VARCHAR(20),
170     business_id VARCHAR(15),
171     timestamp DATETIME,
172     amount DECIMAL(10,2),
173     declined TINYINT(1),
174     product_ids VARCHAR(100),
175     user_id INT,
176     lat FLOAT,
177     longitude FLOAT
178 );
179
```

|      | Time     | Action                                                                                                              | Response                                                 |
|------|----------|---------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------|
| ⚠ 45 | 00:56:03 | CREATE TABLE transactions ( id VARCHAR(255) PRIMARY KEY, card_id VARCHAR(20), business_id VARCHAR(15), timestamp... | 0 row(s) affected, 1 warning(s): 1681 Integer display... |

```
180 • LOAD DATA LOCAL INFILE '/Users/patrycjaproniewska/Documents/transactions.csv'
181 INTO TABLE transactions
182 FIELDS TERMINATED BY ','
183 LINES TERMINATED BY '\n'
184 IGNORE 1 LINES
185 (id, card_id, business_id, timestamp, amount, declined, product_ids, user_id, lat, longitude);
186
```

|      | Time     | Action                                                                                                                            | Response                                             |
|------|----------|-----------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------|
| ✓ 46 | 00:59:59 | LOAD DATA LOCAL INFILE '/Users/patrycjaproniewska/Documents/transactions.csv' INTO TABLE transactions FIELDS TERMINATED BY ','... | 100000 row(s) affected Records: 100000 Deleted: 0... |

```

187 • SELECT *
188 FROM transactions;
189
100% 25:190

```

| id                                   | card_id  | business_id | timestamp           | amount | declined | product_ids    | user_id | lat     | longitude |
|--------------------------------------|----------|-------------|---------------------|--------|----------|----------------|---------|---------|-----------|
| 00045D6B-ED2E-4F2F-8186-CEE074D875D0 | CcS-6699 | b-2390      | 2020-07-14 15:37:45 | 326.01 | 0        | 30, 11, 16, 81 | 2118    | 29.7573 | -95.3796  |
| 000481C3-1C26-4FEF-83A0-4CD0EB004BBD | CcS-6696 | b-2230      | 2017-09-04 19:44:53 | 161.60 | 0        | 72             | 2115    | 53.5489 | -113.503  |
| 00051AA4-9CBE-4268-B070-C38062A1B3E2 | CcS-7606 | b-2266      | 2017-01-05 18:19:25 | 148.91 | 0        | 18             | 3025    | 52.2084 | 5.69081   |
| 0008A312-EDFE-4A4F-BC99-E9C92EC3CA4D | CcU-3358 | b-2598      | 2023-09-23 04:51:43 | 294.59 | 0        | 35, 33, 19     | 215     | 53.5535 | -113.499  |

transactions 9

| Time        | Action                     | Response               |
|-------------|----------------------------|------------------------|
| 47 01:01:08 | SELECT * FROM transactions | 100000 row(s) returned |

Podemos observar en los datos que la columna **product\_ids** no tiene un valor único, sino varios valores dentro de la misma celda. Por ahora no necesito esta columna para las consultas de este ejercicio, pero más adelante voy a resolver este problema.

Establecemos relaciones entre tablas:

```

199 -- crear relacion entre transactions y credit_card (FK constraint)
200 • ALTER TABLE `transactions`
201 ADD CONSTRAINT fk_transactions_credit_card
202 FOREIGN KEY (card_id)
203 REFERENCES credit_card(id)
204 ON UPDATE CASCADE;
205
100% 19:204

```

| Time        | Action                                                                                                                         | Response                                              |
|-------------|--------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------|
| 50 01:04:58 | ALTER TABLE `transactions` ADD CONSTRAINT fk_transactions_credit_card FOREIGN KEY (card_id) REFERENCES credit_card(id) ON U... | 100000 row(s) affected Records: 100000 Duplicates:... |

```

192 -- crear relación entre transactions y company (FK constraint)
193 • ALTER TABLE `transactions`
194 ADD CONSTRAINT fk_transactions_company
195 FOREIGN KEY (business_id)
196 REFERENCES company(company_id)
197 ON UPDATE CASCADE;
198
100% 19:197

```

| Time        | Action                                                                                                                    | Response                                              |
|-------------|---------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------|
| 49 01:02:42 | ALTER TABLE `transactions` ADD CONSTRAINT fk_transactions_company FOREIGN KEY (business_id) REFERENCES company(company... | 100000 row(s) affected Records: 100000 Duplicates:... |



```
206 -- crear relación entre transactions y users
207 • ALTER TABLE `transactions`
208 ADD CONSTRAINT fk_transactions_user
209 FOREIGN KEY (user_id)
210 REFERENCES users(id)
211 ON UPDATE CASCADE;
212
```

100% 19:211

Action Output

|      | Time     | Action                                                                                                                      | Response                                              | Du  |
|------|----------|-----------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------|-----|
| ✓ 51 | 01:06:01 | ALTER TABLE `transactions` ADD CONSTRAINT fk_transactions_user FOREIGN KEY (user_id) REFERENCES users(id) ON UPDATE CASC... | 100000 row(s) affected Records: 100000 Duplicates:... | 1.9 |

▼ transactions\_sprint

▼ Tables

> company

> credit\_card

▼ transactions

> Columns

> Indexes

▼ Foreign Keys

fk\_transactions\_company

fk\_transactions\_credit\_card

fk\_transactions\_user

> Triggers

> users

Views

Stored Procedures

Functions

Object Info

Session

Related Tables:

Target

company (business\_id → company\_id)

On Update

CASCADE

On Delete

RESTRICT

Target

credit\_card (card\_id → id)

On Update

CASCADE

On Delete

RESTRICT

Target

users (user\_id → id)

On Update

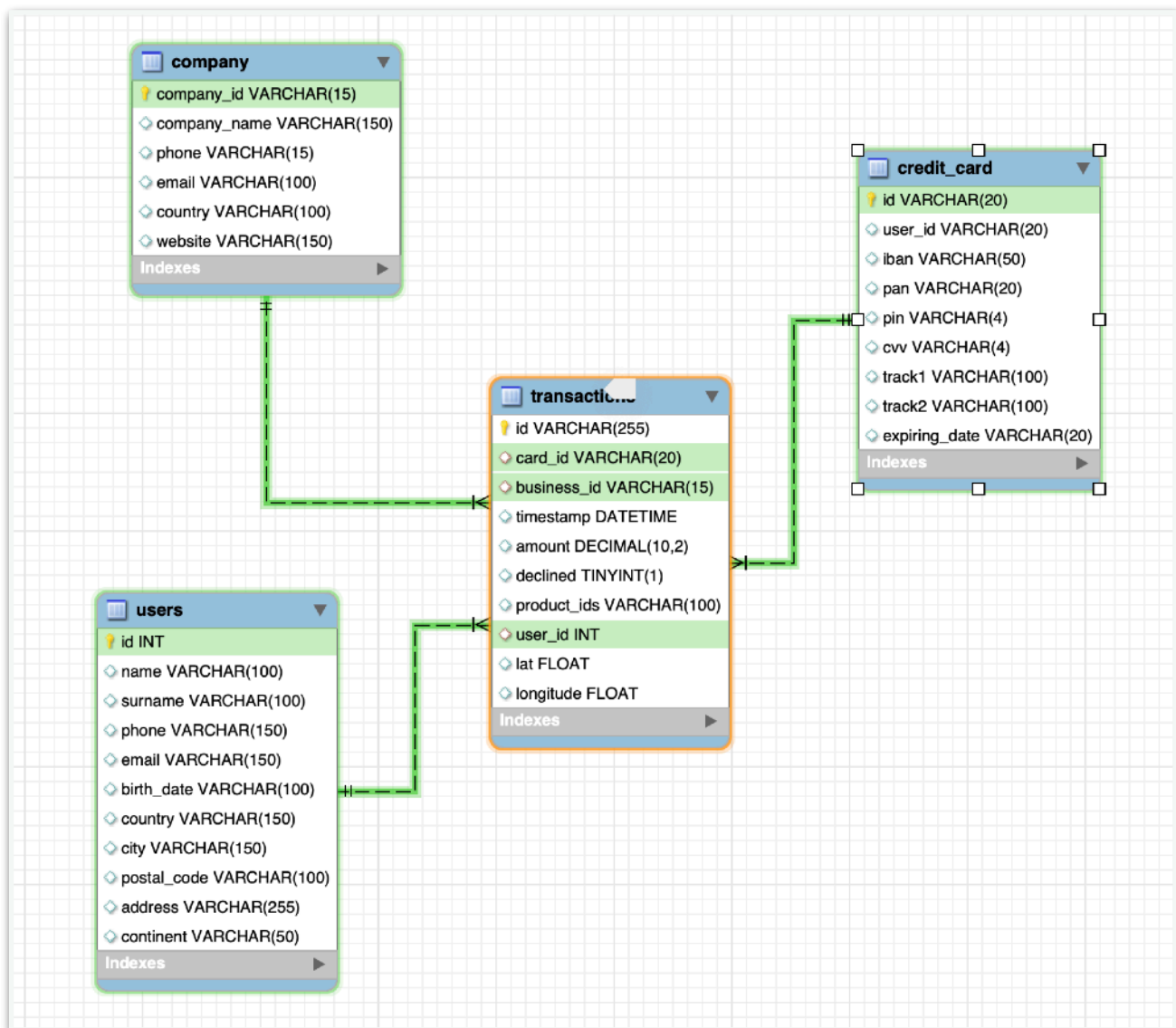
CASCADE

On Delete

RESTRICT



## Diagrama resultante de base de datos:



## Modelo de Base de Datos Tipo Estrella

La base de datos **transactions** está estructurada como un esquema estrella. En este modelo, la tabla **transactions** actúa como la **tabla de hechos**, registrando cada operación realizada, mientras que las tablas **company**, **users** y **credit\_card** funcionan como **tablas de dimensión**, proporcionando información descriptiva sobre las empresas, los usuarios y las tarjetas de crédito asociadas.

## Relaciones:

Las relaciones son de **uno a muchos (1:N)**:

- Una empresa puede tener muchas transacciones, pero cada transacción pertenece a una sola empresa.
- Un usuario puede realizar muchas transacciones, pero cada transacción está asociada a un único usuario.
- Una tarjeta de crédito puede utilizarse en muchas transacciones, pero cada transacción se realiza con una sola tarjeta.

## Tabla de hechos: transactions

Registra los datos numéricos y de eventos que representan cada operación financiera.

- id – identifica de forma única cada transacción (**PK**) / VARCHAR(255)
- card\_id – identificador de la tarjeta de crédito utilizada (**FK**) / VARCHAR(20)
- business\_id – identificador de la empresa asociada (**FK**) / VARCHAR(15)
- user\_id – identificador del usuario que realizó la transacción (**FK**) / INT
- timestamp – fecha y hora de la transacción / DATETIME
- amount – cantidad de dinero en la transacción / DECIMAL(10,2)
- declined – indica si la transacción fue rechazada / TINYINT(1)
- product\_ids – lista de productos involucrados / VARCHAR(100)
- lat – latitud de la ubicación / FLOAT
- longitude – longitud de la ubicación / FLOAT

## Dimensión: company

Guarda la información de las empresas.

- company\_id – identifica de forma única cada empresa (**PK**) / VARCHAR(15)

- company\_name – nombre de la empresa / VARCHAR(150)
- phone – teléfono / VARCHAR(15)
- email – correo electrónico / VARCHAR(100)
- country – país donde opera la empresa / VARCHAR(100)
- website – página web / VARCHAR(150)

### **Dimensión: credit\_card**

Almacena los datos esenciales de las tarjetas de crédito.

- id – identifica de forma única cada tarjeta (**PK**) / VARCHAR(20)
- user\_id – identificador del usuario titular / VARCHAR(20)
- iban – número IBAN de la cuenta / VARCHAR(50)
- pan – número de la tarjeta / VARCHAR(20)
- pin – código PIN de seguridad / VARCHAR(4)
- cvv – código de verificación / VARCHAR(4)
- track1, track2 – datos de pista magnética / VARCHAR(100)
- expiring\_date – fecha de caducidad / VARCHAR(20)

### **Dimensión: users**

Contiene la información descriptiva de los usuarios.

- id – identifica de forma única a cada usuario (**PK**) / INT
- name – nombre / VARCHAR(100)
- surname – apellido / VARCHAR(100)
- phone – teléfono / VARCHAR(150)
- email – correo electrónico / VARCHAR(150)
- birth\_date – fecha de nacimiento / VARCHAR(100)
- country – país / VARCHAR(150)

- city – ciudad / VARCHAR(150)
- postal\_code – código postal / VARCHAR(100)
- address – dirección / VARCHAR(255)
- continent – continente / VARCHAR(50)

## Relaciones FK:

En este esquema, la tabla **transactions** es la **tabla central (de hechos)** que conecta las tres dimensiones mediante claves foráneas.

- transactions.business\_id → company.company\_id
- transactions.card\_id → credit\_card.id
- transactions.user\_id → users.id

## Ejercicio 1

Realiza una subconsulta que muestre a todos los usuarios con más de 80 transacciones utilizando al menos 2 tablas.

The screenshot shows a SQL IDE interface. The query editor contains the following SQL code:

```

232 • SELECT u.name, u.surname, u.email
233 FROM users AS u
234 WHERE EXISTS (
235     SELECT 1
236     FROM transactions AS t
237     WHERE t.user_id = u.id
238     GROUP BY t.user_id
239     HAVING COUNT(t.id) > 80
240 );
241

```

Below the query editor, the 'Result Grid' shows the results of the query. It displays a table with 4 rows and 3 columns: name, surname, and email. The first four rows are visible:

| name   | surname   | email                        |
|--------|-----------|------------------------------|
| Molly  | Gilliam   | donec@outlook.couk           |
| Dxwgi  | Hwcru     | dxwgi.hwcru@example.com      |
| Bnyr   | Astuw     | bnyr.astuw@example.com       |
| Sfzzoh | Xgvfridxs | sfzzoh.xgvfridxs@example.com |

At the bottom of the result grid, it says 'users 4'. Below the result grid, the 'Action Output' section shows the execution details:

|    | Time     | Action                                                                                                                    | Response          |
|----|----------|---------------------------------------------------------------------------------------------------------------------------|-------------------|
| 10 | 00:05:27 | SELECT u.name, u.surname, u.email FROM users AS u WHERE EXISTS ( SELECT 1 FROM transactions AS t WHERE t.user_id = u.i... | 4 row(s) returned |

## Ejercicio 2

Muestra la media de amount por IBAN de las tarjetas de crédito en la compañía Donec Ltd., utiliza por lo menos 2 tablas.

```
250 • SELECT cc.iban, AVG(t.amount) AS average_amount
251 FROM transactions AS t
252 JOIN credit_card AS cc
253 ON t.card_id = cc.id
254 JOIN company AS co
255 ON t.business_id = co.company_id
256 WHERE co.company_name = 'Donec Ltd'
257 GROUP BY cc.iban;
```

100% 1:244

Result Grid Filter Rows: Search Export:

| iban                         | average_amount |
|------------------------------|----------------|
| XX911406401125586307586805   | 356.246667     |
| SK9446370242474562577506     | 142.960000     |
| XX776752917845952975555640   | 257.370000     |
| XX413827362289719304908990   | 139.590000     |
| XX347787246070769610780308   | 240.410000     |
| XX688768436543090894854602   | 188.580000     |
| MK28368851538688349          | 439.390000     |
| PL76249283566852676343404576 | 541.560000     |
| XX804008331134918341803913   | 189.210000     |

Result 11

Action Output

|      | Time     | Action                                                                                                                            | Response            |
|------|----------|-----------------------------------------------------------------------------------------------------------------------------------|---------------------|
| ✓ 21 | 00:16:36 | SELECT cc.iban, AVG(t.amount) AS average_amount FROM transactions AS t JOIN credit_card AS cc ON t.card_id = cc.id JOIN compan... | 371 row(s) returned |

## Nivel 2

Crea una nueva tabla que refleje el estado de las tarjetas de crédito basado en si las tres últimas transacciones han sido declinadas entonces es *inactivo*, si al menos una no es rechazada entonces es *activo*. Partiendo de esta tabla responde:

```
385 -- 1 Crear la tabla donde guardaremos el estado de cada tarjeta
386 • CREATE TABLE credit_card_status (
387     card_id VARCHAR(20) PRIMARY KEY,
388     status VARCHAR(10)
389 );
390
```

100% 3:389

Action Output

|       | Time     | Action                                                          | Response          |
|-------|----------|-----------------------------------------------------------------|-------------------|
| ✓ 100 | 13:24:09 | CREATE TABLE credit_card_status ( card_id VARCHAR(20) PRIMAR... | 0 row(s) affected |

```

391  -- Usamos una función de ventana (ROW_NUMBER())
392  -- para numerar las transacciones de cada tarjeta (card_id),
393  -- ordenadas por fecha (timestamp) de más reciente a más antigua.
394  -- despues tendremos que hacer un filtro WHERE para. mostrar ultimas 3
395  • SELECT t.card_id, t.declined,
396     ROW_NUMBER() OVER (PARTITION BY t.card_id ORDER BY t.timestamp DESC) AS ranking_transaccion
397 FROM transactions AS t;
398
399
100% 24:397

```

**Result Grid** Filter Rows: Search Export: Fetch rows:

| card_id  | declined | ranking_transacci... |
|----------|----------|----------------------|
| CcS-4857 | 0        | 3                    |
| CcS-4857 | 1        | 4                    |
| CcS-4857 | 0        | 5                    |
| CcS-4857 | 0        | 6                    |
| CcS-4857 | 0        | 7                    |
| CcS-4857 | 0        | 8                    |
| CcS-4857 | 0        | 9                    |
| CcS-4857 | 0        | 10                   |
| CcS-4857 | 0        | 11                   |

Result 39

**Action Output**

| Time     | Action                                                             | Response               |
|----------|--------------------------------------------------------------------|------------------------|
| 14:56:21 | SELECT t.card_id, t.declined, ROW_NUMBER() OVER (PARTITION BY t... | 100000 row(s) returned |

Ahora tenemos insertar los datos a la tabla **credit\_card\_status** que habíamos creado antes, con CASE (condición) y Window Function, filtrando con WHERE para que la tabla solo refleje las ultimas 3 transacciones:

```

406 • INSERT INTO credit_card_status (card_id, status)
407 SELECT
408     card_id,
409     CASE
410         WHEN COUNT(*) >= 3 AND SUM(declined) = COUNT(*) THEN 'Inactivo'
411         ELSE 'Activo'
412     END AS status
413 FROM (
414     SELECT t.card_id, t.declined,
415     ROW_NUMBER() OVER (PARTITION BY t.card_id ORDER BY t.timestamp DESC) AS ranking_transaccion
416     FROM transactions AS t) AS ultimas_transacciones
417 WHERE ranking_transaccion <= 3
418 GROUP BY card_id;
419
100% 18:418

```

**Action Output**

| Time     | Action                                                              | Response                                                     |
|----------|---------------------------------------------------------------------|--------------------------------------------------------------|
| 15:08:13 | INSERT INTO credit_card_status (card_id, status) SELECT card_id,... | 5000 row(s) affected Records: 5000 Duplicates: 0 Warnings: 0 |

```

293 -- relacionamos la tabla nueva con tabla credit_card:
294 • ALTER TABLE credit_card_status
295   ADD CONSTRAINT fk_credit_card_status_card_id
296   FOREIGN KEY (card_id)
297   REFERENCES credit_card(id)
298   ON UPDATE CASCADE
299   ON DELETE CASCADE;
300
301 -- CASCADE automaticamente borra o cambia datos en credit_card_status
302 -- cuando cambian en credit_card

```

100% 19:299

Action Output

|     | Time     | Action                                                                                       | Response                                                     |
|-----|----------|----------------------------------------------------------------------------------------------|--------------------------------------------------------------|
| 133 | 12:17:14 | ALTER TABLE credit_card_status ADD CONSTRAINT fk_credit_card_status_card_id FOREIGN KEY (... | 5000 row(s) affected Records: 5000 Duplicates: 0 Warnings: 0 |

## Ejercicio 1

¿Cuántas tarjetas están activas?

```

295 -- ¿Cuántas tarjetas están activas?
296
297 • SELECT COUNT(ccs.card_id)
298   FROM credit_card_status AS ccs
299   WHERE status = 'Activo';
300
301

```

100% 25:299

Result Grid

| COUNT(ccs.card_id) |
|--------------------|
| 4995               |

Result 47

Action Output

|     | Time     | Action                                                                           | Response          |
|-----|----------|----------------------------------------------------------------------------------|-------------------|
| 126 | 11:40:42 | SELECT COUNT(ccs.card_id) FROM credit_card_status AS ccs WHERE status = 'Activo' | 1 row(s) returned |

## Nivel 3

Crea una tabla con la que podamos unir los datos del nuevo archivo products.csv con la base de datos creada, teniendo en cuenta que desde transaction tienes product\_ids. Genera la siguiente consulta:

## Ejercicio 1

Necesitamos conocer el número de veces que se ha vendido cada producto.



Creamos tabla **products** e importamos datos del archivo csv:

```
307 • CREATE TABLE products (  
308     id INT PRIMARY KEY,  
309     product_name VARCHAR(150),  
310     price DECIMAL(10,2),  
311     colour VARCHAR(10),  
312     weight DECIMAL(4,1),  
313     warehouse_id VARCHAR(10)  
314 );  
315
```

100% 3:314

Action Output

|      | Time     | Action                                                                                             | Response          |
|------|----------|----------------------------------------------------------------------------------------------------|-------------------|
| ✓ 56 | 01:49:48 | CREATE TABLE products ( id INT PRIMARY KEY, product_name VARCHAR(150), price DECIMAL(10,2), col... | 0 row(s) affected |

Error al cargar los datos (Incorrect decimal value):

```
316 • LOAD DATA LOCAL INFILE '/Users/patrycjaproniewska/Documents/products.csv'  
317 INTO TABLE products  
318 FIELDS TERMINATED BY ','  
319 ENCLOSED BY ''''  
320 LINES TERMINATED BY '\n'  
321 IGNORE 1 LINES  
322 (id, product_name, price, colour, weight, warehouse_id);  
323
```

100% 25:312

Action Output

|      | Time     | Action                                                                          | Response                                                                                                         |
|------|----------|---------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------|
| ⚠ 58 | 01:54:07 | LOAD DATA LOCAL INFILE '/Users/patrycjaproniewska/Documents/products.csv' IN... | 100 row(s) affected, 100 warning(s): 1366 Incorrect decimal value: '\$161.11' for column 'price' at row 1 136... |

Los precios se han perdido:

```
324 • SELECT * FROM products;  
325
```

100% 24:324

Result Grid

|     | id | product_name           | price | colour  | weight | warehouse_id |
|-----|----|------------------------|-------|---------|--------|--------------|
| ▶ 1 | 1  | Direwolf Stannis       | 0.00  | #7c7c7c | 1.0    | WH-4         |
| ▶ 2 | 2  | Tarly Stark            | 0.00  | #919191 | 2.0    | WH-3         |
| ▶ 3 | 3  | duel tourney Lannister | 0.00  | #d8d8d8 | 1.5    | WH-2         |
| ▶ 4 | 4  | warden south duel      | 0.00  | #111111 | 3.0    | WH-1         |
| ▶ 5 | 5  | clawhammer             | 0.00  | #d8d8d8 | 2.0    | WH-4         |

products 13

Action Output

|      | Time     | Action                 | Response            |
|------|----------|------------------------|---------------------|
| ✓ 59 | 01:57:47 | select * from products | 100 row(s) returned |

El archivo CSV tenía un símbolo de \$ antes de cada precio, por eso aparecía el error. Tenemos que limpiar el precio y borrar el símbolo de \$.

```
320 -- Importar los datos del CSV y eliminar el símbolo "$" de la columna price
321 -- antes de insertarla en la tabla usando REPLACE(@price, '$', '').
322 • LOAD DATA LOCAL INFILE '/Users/patrycjaproniewska/Documents/products.csv'
323 INTO TABLE products
324 FIELDS TERMINATED BY ','
325 ENCLOSED BY '"'
326 LINES TERMINATED BY '\n'
327 IGNORE 1 LINES
328 (id, product_name, @price, colour, weight, warehouse_id)
329 SET price = REPLACE(@price, '$', '');|
```

|                                                                                                  |          |
|--------------------------------------------------------------------------------------------------|----------|
| 100%                                                                                             | 38:329   |
| Action Output                                                                                    |          |
| 63                                                                                               | 02:14:04 |
| LOAD DATA LOCAL INFILE '/Users/patrycjaproniewska/Documents/products.csv' INTO TABLE products... |          |
| 100 row(s) affected Records: 100 Deleted: 0 Skipped: 0 Warnings: 0                               |          |

@price - variable temporal ('\$161.11')

price - columna real de tu tabla (DECIMAL(10,2))

```
327 -- ponemos $ en el nombre de la columna para mantener esta info:
328 • ALTER TABLE products
329 RENAME COLUMN price TO price_$USD;
330
```

|                                                         |          |
|---------------------------------------------------------|----------|
| 100%                                                    | 35:329   |
| Action Output                                           |          |
| 130                                                     | 12:00:34 |
| ALTER TABLE products RENAME COLUMN price TO price_\$USD |          |
| 0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0  |          |

```
331 • SELECT * FROM products;|
332
```

100%

↕

24:331

Result Grid

Filter Rows:

Search

Edit:

Export/Import:

|             | id | product_name           | price_\$USD | colour  | weight | warehouse_id |  |
|-------------|----|------------------------|-------------|---------|--------|--------------|--|
| ▶           | 1  | Direwolf Stannis       | 161.11      | #7c7c7c | 1.0    | WH-4         |  |
|             | 2  | Tarly Stark            | 9.24        | #919191 | 2.0    | WH-3         |  |
|             | 3  | duel tourney Lannister | 171.13      | #d8d8d8 | 1.5    | WH-2         |  |
|             | 4  | warden south duel      | 71.89       | #111111 | 3.0    | WH-1         |  |
| products 48 |    |                        |             |         |        |              |  |

Action Output

↕

|   |     | Time     | Action                 | Response            |
|---|-----|----------|------------------------|---------------------|
| ✔ | 131 | 12:03:31 | SELECT * FROM products | 100 row(s) returned |

En tabla **transactions**, tenemos una columna **product\_ids** que contiene varios IDs (identificadores) dentro de una celda. Esto crea un problema en nuestra base relacional - la relación con tabla **transactions** no se puede establecer. Necesitamos mantener regla básica del modelo relacional, que cada celda contenga un único valor. Por esta razón voy a crear tabla intermedia **transaction\_product**. En esta tabla se almacenará una fila por cada relación entre transacción y producto. Después, voy a insertar los datos en la nueva tabla intermedia usando la función **JSON\_TABLE**.

Creamos tabla intermedia **transaction\_product** para conectar bien el modelo:

```
295  -- para conectar Product con Transaction, creo una tabla intermedia:
296  -- transaction_product:
297
298  CREATE TABLE transaction_product (
299      transaction_id VARCHAR(255),
300      product_id INT,
301      FOREIGN KEY (transaction_id) REFERENCES transactions(id) ON DELETE CASCADE,
302      FOREIGN KEY (product_id) REFERENCES products(id) ON DELETE CASCADE
303  );
```

100% 3:303

Action Output

|    | Time     | Action                                                                                     | Response          |
|----|----------|--------------------------------------------------------------------------------------------|-------------------|
| 72 | 12:14:50 | CREATE TABLE transaction_product ( transaction_id VARCHAR(255), product_id INT, FOREIGN... | 0 row(s) affected |

```

343 -- t.product_id - es u. texto con comas, no estan separadas: Ej.: "75, 73, 98"
344 -- este proceso funciona de la siguiente manera:
345 -- podemos fabricar una cadena con formato JSON a partir de product_ids (funciona con STRING)
346 -- luego JSON_TABLE la divide/explota en filas y de estamaneira podemos separar los product_id
347 -- Comprobar el resultado de separar los product_id de producto para que en transaction_product
348 -- aparezcan una fila por cada producto:

```

```

350 • SELECT
351     t.id AS transaction_id,
352     CAST(p.product_id AS UNSIGNED) AS product_id
353 FROM transactions AS t,
354 JSON_TABLE(
355     CONCAT(
356         '["',
357         REPLACE(REPLACE(t.product_ids, ' ', ''), ', ', '"', ''),
358         '"]'
359     ),
360     COLUMNS (product_id VARCHAR(10) PATH '$')
361 ) AS p;

```

100% 8:361

Result Grid Filter Rows: Search Export: Fetch rows:

| transaction_id                       | product_id |
|--------------------------------------|------------|
| 00043A49-2949-494B-A5DD-A5BAE3BB19DD | 16         |
| 00043A49-2949-494B-A5DD-A5BAE3BB19DD | 26         |
| 00043A49-2949-494B-A5DD-A5BAE3BB19DD | 97         |
| 00043A49-2949-494B-A5DD-A5BAE3BB19DD | 87         |
| 000447FF-B650-4DCF-85DE-C7ED0FE1CAAD | 66         |

Result 49

Action Output

| Time         | Action                                                                                    | Response               |
|--------------|-------------------------------------------------------------------------------------------|------------------------|
| 132 12:06:33 | SELECT t.id AS transaction_id, CAST(p.product_id AS UNSIGNED) AS product_id FROM trans... | 253391 row(s) returned |

\* AS UNSIGNED convierte los valores de texto a números enteros positivos.

```

324 -- insertamos los datos en la tabla intermedia usando la función JSON_TABLE:

```

```

326 • INSERT INTO transaction_product (transaction_id, product_id)
327 SELECT
328     t.id,
329     CAST(p.product_id AS UNSIGNED)
330 FROM transactions AS t
331 CROSS JOIN JSON_TABLE(
332     CONCAT(
333         '["',
334         REPLACE(REPLACE(t.product_ids, ' ', ''), ', ', '"', ''),
335         '"]'
336     ),
337     '[$*]'
338     COLUMNS (
339         product_id VARCHAR(10) PATH '$'
340     )
341 ) AS p;

```

100% 8:341

Action Output

| Time        | Action                                                                                           | Response                                                         |
|-------------|--------------------------------------------------------------------------------------------------|------------------------------------------------------------------|
| 78 13:59:07 | INSERT INTO transaction_product (transaction_id, product_id) SELECT t.id, CAST(p.product_id A... | 253391 row(s) affected Records: 253391 Duplicates: 0 Warnings: 0 |

```

344 -- visualizamos la tabla nueva con datos insertados:
345 • SELECT *
346 FROM transaction_product;
347

```

100% 26:346

**Result Grid** Filter Rows: Search Export: Fetch rows:

| transaction_id                       | product_id |
|--------------------------------------|------------|
| 00043A49-2949-494B-A5DD-A5BAE3BB1... | 16         |
| 00043A49-2949-494B-A5DD-A5BAE3BB1... | 26         |
| 00043A49-2949-494B-A5DD-A5BAE3BB1... | 97         |
| 00043A49-2949-494B-A5DD-A5BAE3BB1... | 87         |
| 000447FE-B650-4DCF-85DE-C7ED0EE1...  | 66         |
| 000447FE-B650-4DCF-85DE-C7ED0EE1...  | 66         |

transaction\_product 25

**Action Output**

|    | Time     | Action                            | Response               |
|----|----------|-----------------------------------|------------------------|
| 79 | 14:00:29 | SELECT * FROM transaction_product | 253391 row(s) returned |

Ahora podemos relacionar las tablas a través de foreign key:

```

349 -- creamos la relación entre transaction_product y transactions
350 • ALTER TABLE transaction_product
351 ADD CONSTRAINT fk_transactionproduct_transaction
352 FOREIGN KEY (transaction_id) REFERENCES transactions(id)
353 ON DELETE CASCADE;
354

```

100% 19:353

**Action Output**

|    | Time     | Action                                                                                                                                                       | Response                                                         |
|----|----------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------|
| 80 | 14:09:54 | ALTER TABLE transaction_product ADD CONSTRAINT fk_transactionproduct_transaction FOREIGN KEY (transaction_id) REFERENCES transactions(id) ON DELETE CASCADE; | 253391 row(s) affected Records: 253391 Duplicates: 0 Warnings: 0 |

```

356 -- creamos la relación entre transaction_product y products
357 • ALTER TABLE transaction_product
358 ADD CONSTRAINT fk_transactionproduct_product
359 FOREIGN KEY (product_id) REFERENCES products(id)
360 ON DELETE CASCADE;
361

```

100% 19:360

**Action Output**

|    | Time     | Action                                                                                                                                           | Response                                                         |
|----|----------|--------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------|
| 83 | 14:12:18 | ALTER TABLE transaction_product ADD CONSTRAINT fk_transactionproduct_product FOREIGN KEY (product_id) REFERENCES products(id) ON DELETE CASCADE; | 253391 row(s) affected Records: 253391 Duplicates: 0 Warnings: 0 |

```

361 -- definimos los dos campos como PK (clave compuesta y FK a la vez)
362 • ALTER TABLE transaction_product
363 ADD PRIMARY KEY (transaction_id, product_id);
364

```

100% 1:365

**Action Output**

|    | Time     | Action                                                                        | Response                                               |
|----|----------|-------------------------------------------------------------------------------|--------------------------------------------------------|
| 86 | 14:41:19 | ALTER TABLE transaction_product ADD PRIMARY KEY (transaction_id, product_id); | 0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0 |

Entre transactions y products existe una relación muchos-a-muchos:

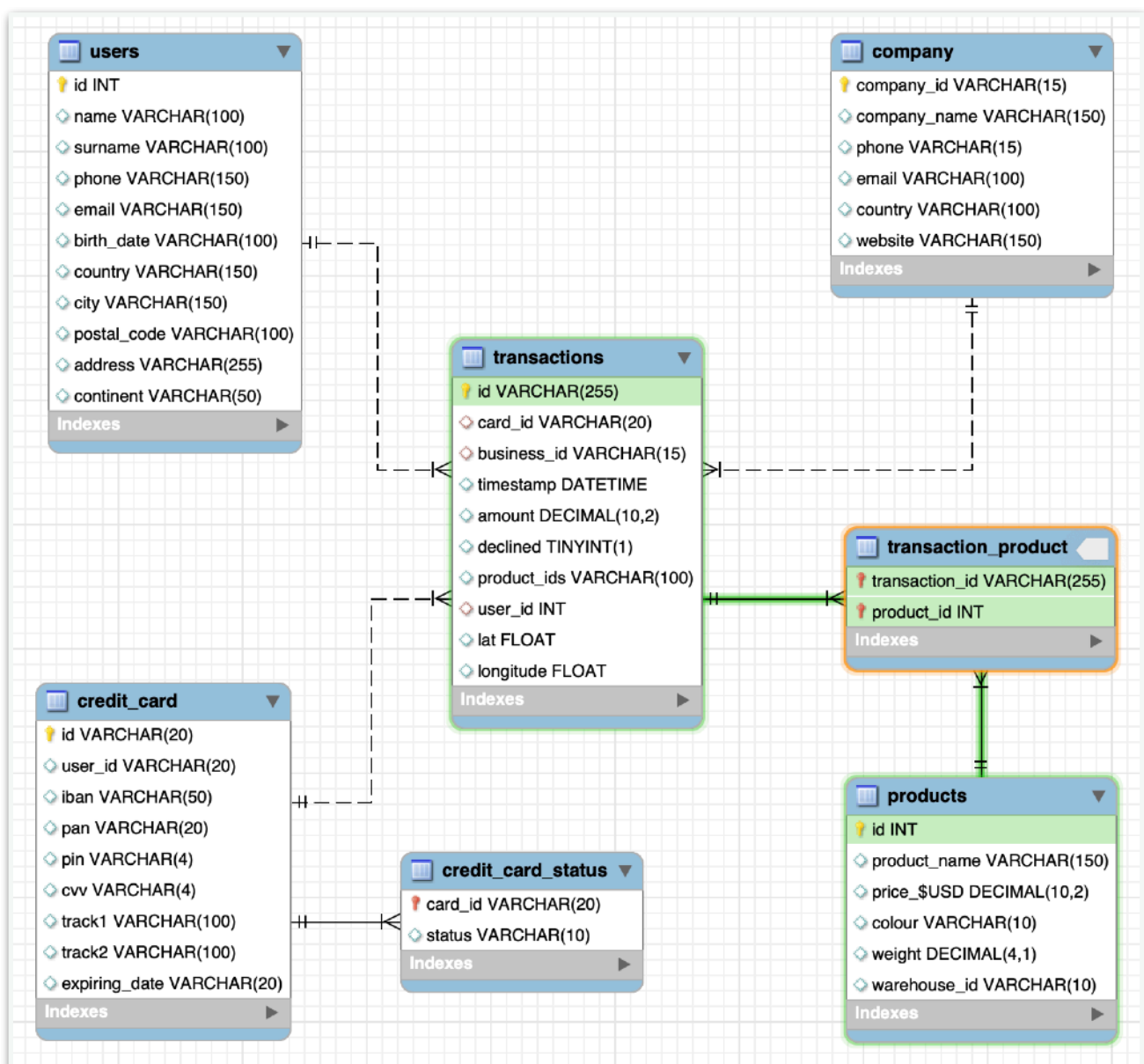
- una transacción puede tener muchos productos,
- un producto puede aparecer en muchas transacciones.

Las bases de datos no pueden crear directamente una relación muchos-a-muchos, por eso usamos una tabla intermedia: transaction\_product.

Esa tabla no es una entidad independiente, sino una tabla “puente” o “intermedia” que conecta las otras dos. Por eso, en tabla transaction\_product necesitamos crear los FK para conectarla con otras tablas: transaction y products. Así la tabla intermedia apunta a ellas y empieza a ser parte de modelo.

\* Clave compuesta = clave primaria formada por varias columnas. Ninguna de esas columnas por separado identifica de forma única una fila, pero la combinación de las dos sí.

Ahora el diagrama se muestra así:



# Ejercicio 1

Necesitamos conocer el número de veces que se ha vendido cada producto.

```
424 • SELECT
425     p.product_name, p.id AS product_id,
426     COUNT(tp.product_id) AS veces_vendido
427 FROM products AS p
428 JOIN transaction_product AS tp
429     ON p.id = tp.product_id
430 JOIN transactions AS t
431     ON t.id = tp.transaction_id
432 WHERE t.declined = 0
433 GROUP BY p.product_name, p.id
434 ORDER BY veces_vendido DESC;
435
```

100% 29:434

**Result Grid** Filter Rows: Search Export:

| product_name        | product_id | veces_vendido |
|---------------------|------------|---------------|
| riverlands the duel | 52         | 2642          |
| Tully maester Taryl | 29         | 2627          |
| duel Direwolf       | 21         | 2603          |
| the duel warden     | 16         | 2602          |
| duel warden         | 33         | 2593          |

Result 51

Action Output

|       | Time     | Action                                                                                   | Response            |
|-------|----------|------------------------------------------------------------------------------------------|---------------------|
| ✓ 143 | 13:32:02 | SELECT p.product_name, p.id AS product_id, COUNT(tp.product_id) AS veces_vendido FROM... | 100 row(s) returned |